

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICHE,
INFORMATICHE E FISICHE

IBML - Internet of Things, Big Data, Machine Learning



Internet of Things Progetto Pet Tracker

Autori

Andrea Gioia, 169484
Alessio Nicodemo, 166972
Mattia Nadal, 167372
Francesco Sabot, 167339

169484@spes.uniud.it
166972@spes.uniud.it
167372@spes.uniud.it
167339@spes.uniud.it

Sommario

Il progetto Pet Tracker propone la realizzazione di un sistema per il tracciamento in tempo reale di animali domestici di taglia media o grande, tramite un dispositivo *embedded* indossabile. L'obiettivo del lavoro è sviluppare una soluzione affidabile, scalabile e a basso consumo per il monitoraggio continuo dell'animale.

La soluzione si basa su una piattaforma ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere lo stato della batteria del dispositivo, insieme alla posizione geografica e all'attività motoria dell'animale, verso un *backend* remoto. L'architettura del sistema segue il modello IoT World Forum (IoTWF) e integra diverse tecniche di ottimizzazione energetica, tra cui l'utilizzo delle modalità *Deep Sleep* e *Hot Start*. I dati raccolti vengono elaborati dal *backend*, resi accessibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

Indice

1	Architettura del sistema	4
1.1	Mappatura sul modello IoTWF	4
2	Hardware	4
2.1	LilyGo T-SIM7670G-S3	4
2.2	Antenna GPS e LTE	4
2.3	Batteria e Power Management	4
2.4	BMA400 CJMCU-400	5
3	Firmware: Logica di funzionamento	5
3.1	Gestione della Persistenza e Hot Start	5
3.2	Protocollo di Comunicazione HTTPS	5
3.3	Risparmio Energetico e Deep Sleep	6
3.4	Monitoraggio Hardware e Batteria	6
3.5	Acquisizione GPS e Timeout	6
3.6	Connessione alla Rete LTE	7
4	Back-end e Database	7
4.1	Architettura di Hosting e Network Exposure	7
4.2	Sistema di Backup e Data Retention	7
4.3	Monitoraggio e Business Continuity	7
4.4	Struttura dei dati e pulizia automatica	8
4.5	Interfaccia API e API Rules	8
4.5.1	API Rules e Sicurezza	8
4.6	Smistamento e Normalizzazione	8
4.6.1	Ottimizzazione delle query: lettura centralizzata della board	9
4.6.2	Funzionamento del Trigger onRecordAfterCreateSuccess	9
4.6.3	Macchina a stati delle attività	9
4.6.4	Gestione del Risveglio e Deduplicazione delle Sessioni	10
4.6.5	Watchdog di Inattività e Split Notturmo	10
4.6.6	Vantaggi Architettureali e Corrispondenza IoTWF	11
5	Front-end e Applicazione Mobile	11
5.1	Architettura a livelli	11
5.2	Autenticazione e persistenza della sessione	12
5.3	Splash Screen e Caricamento Parallelo	12
5.4	Comunicazione in Tempo Reale	12
5.5	Geofencing e Geocoding	13
5.6	Schermate e Funzionalità Principali	13
5.6.1	Analisi delle Attività	13
5.6.2	Mappe in Tempo Reale	14
5.6.3	Gestione delle Zone Sicure	15
6	Sistema di Notifiche Push	15
6.1	Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase	15
6.2	Architettura del FCM Bridge	16
6.3	Logica di Invio e Trigger	16

6.4	Permessi e Localizzazione	17
7	Vincoli di Sistema	17
7.1	Vincoli Hardware e Ambientali	17
7.2	Vincoli Architettureali e di Concorrenza	18
7.3	Vincoli di Integrazione e Dati Esterni	18
8	Implementazioni Future	19
8.1	Stato della batteria	19
8.2	Database, Infrastruttura Cloud e Gestione della Concorrenza	19
8.3	Ingegnerizzazione e Costruzione della Scheda Hardware	19
9	Conclusioni	20

1 Architettura del sistema

L'architettura del sistema Pet Tracker segue il modello a sette livelli dell'IoTWF, garantendo una transizione strutturata dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IoTWF

- **Livello 1 (Physical Devices):** rappresentato dalla scheda LilyGo e dai sensori GPS e accelerometro.
- **Livello 2 (Connectivity):** gestito tramite il modem LTE e protocolli di comunicazione sicuri.
- **Livello 3 (Edge Computing):** elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** persistenza dei dati sul server PocketBase basato su database SQLite.
- **Livello 5 (Data Abstraction):** pubblicazione dei dati tramite API REST in formato JSON.
- **Livello 6 (Application):** applicazione mobile per la visualizzazione delle informazioni e della posizione dell'animale su mappa.
- **Livello 7 (Collaboration and Processes):** interazione dell'utente con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LilyGo T-SIM7670G-S3

Il dispositivo utilizza una board LilyGo T-SIM7670G-S3 [[lilygo_specs](#)] basata sul modulo **ESP32-S3** dual-core, che gestisce la logica applicativa e la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [[LTE](#)] per la connettività dati e un'antenna ceramica attiva [[GPS](#)] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [**bma400**] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente Arduino per il SoC ESP32-S3, gestisce l'acquisizione dei dati dai sensori, il controllo del modem LTE e l'ottimizzazione energetica tramite una macchina a stati finiti.

3.1 Gestione della Persistenza e Hot Start

Per ridurre drasticamente il Time To First Fix (TTFF) del modulo GNSS, il firmware sfrutta la memoria RTC (Real-Time Clock) dell'ESP32. Questa memoria permette di mantenere i dati vitali anche durante i cicli di Deep Sleep. Le variabili critiche, come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`), vengono dichiarate con l'attributo `RTC_DATA_ATTR`.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente.
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`.
- Sincronizza l'orario del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità Hot Start, che minimizza il tempo di ricerca dei satelliti e, di conseguenza, il consumo energetico.

3.2 Protocollo di Comunicazione HTTPS

La trasmissione dei dati al backend PocketBase è affidata a una serie di comandi AT inviati via seriale al modem SIM7670G. Il firmware incapsula le informazioni in un oggetto JSON strutturato, inviato tramite una richiesta HTTPS POST. Il payload include:

- `board_id`: un identificativo univoco derivato dall'IMEI del modem;
- `geo`: un oggetto con latitudine e longitudine per la gestione cartografica;
- `battery` / `battery_percent` / `charging`: telemetria sullo stato di carica della batteria;
- `steps`: conteggio dei passi rilevato dall'accelerometro nella sessione corrente;
- `sleep`: flag booleano che segnala al server l'imminente entrata in stato di inattività;
- `trip`: flag booleano che segnala al server che l'animale si trova su un veicolo in movimento. Quando attivo, ha priorità assoluta sulla logica di geofencing e sopprime il Deep Sleep per garantire la continuità del tracciamento.

La sequenza di invio si avvale dei comandi AT `AT+HTTPINIT`, `AT+HTTPPARA` e `AT+HTTPDATA` per configurare la sessione HTTP, inserire i dati e infine avviare la trasmissione con

AT+HTTPACTION=1. Al termine di ogni invio, la sessione viene chiusa con AT+HTTPTERM per liberare le risorse del modem.

3.3 Risparmio Energetico e Deep Sleep

Il sistema è progettato per massimizzare l'autonomia della batteria da 3000mAh. La logica di risparmio energetico è coordinata con l'accelerometro BMA400 e tiene conto dello stato di viaggio:

1. **Rilevamento Attività:** il firmware monitora costantemente il numero di passi. Ogni nuovo movimento resetta un timer di inattività (`lastActivityTime`).
2. **Modalità Viaggio:** se il flag `trip` è attivo (velocità GPS superiore a 10 km/h), il Deep Sleep viene soppresso indipendentemente dall'inattività rilevata dall'accelerometro. Il dispositivo continua a trasmettere posizione e telemetria a intervalli regolari per tutto il tempo in cui è in viaggio.
3. **Invio Finale:** se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi) e il flag `trip` non è attivo, il sistema invia un ultimo pacchetto dati aggiornando il flag `sleep` a `true`.
4. **Ibernazione:** il microcontrollore entra in Deep Sleep, spegnendo i moduli radio tramite i comandi `AT+CGNSSPWR=0` e `AT+CPOWD=1`. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dal BMA400 solo al rilevamento di un nuovo movimento.

3.4 Monitoraggio Hardware e Batteria

Il firmware gestisce in modo intelligente l'alimentazione per evitare letture errate o consumi parassiti:

- **Controllo ADC:** il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`. Il valore grezzo viene mediato su 10 campionamenti consecutivi e scalato con fattore 2 per compensare il partitore. La percentuale di carica viene calcolata mediante interpolazione lineare nell'intervallo compreso tra 3,4 V e 4,2 V, corrispondente rispettivamente allo 0% e al 100% di carica.
- **Stato USB:** attraverso la lettura del registro hardware `USB_SERIAL_JTAG_EP1_CONF_REG` dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, settando dinamicamente il campo `charging` nel pacchetto dati. In stato di ricarica, i valori di tensione e percentuale non vengono aggiornati per evitare misurazioni distorte dalla corrente di carica in ingresso; vengono invece restituiti gli ultimi valori validi salvati in memoria RTC.

3.5 Acquisizione GPS e Timeout

Il modulo GNSS SIM7670G viene interrogato tramite il comando `AT AT+CGNSSINFO`. La risposta viene analizzata campo per campo, estraendo latitudine, longitudine e velocità in formato già decimale, senza necessità di conversione dal formato NMEA gradi-minuti. Il firmware attende fino a un massimo di `GPS_TIMEOUT` (180 secondi) per ottenere un fix

valido; superato tale limite senza una posizione acquisita, il ciclo corrente viene saltato e il sistema ritenta al loop successivo. Una volta ottenuto un fix valido, le coordinate e il timestamp vengono salvati in memoria RTC per abilitare il Hot Start al ciclo successivo.

3.6 Connessione alla Rete LTE

All'avvio, il firmware tenta di registrarsi alla rete LTE attraverso la funzione `connectToNetworkFast()`. Vengono configurati la modalità radio (`AT+CNMP=38` per LTE only), il profilo APN dell'operatore e l'attivazione del contesto dati con `AT+CNACT=0,1`. La registrazione viene verificata in polling tramite il comando `AT+CREG?`, controllando la presenza dei codici di stato `0,1` (registrato in rete home) o `0,5` (roaming). Se entro `NET_TIMEOUT` (60 secondi) la registrazione non viene completata, il dispositivo entra direttamente in Deep Sleep per evitare consumi inutili in assenza di copertura.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [`pocketbase`], una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [`ngrok`]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase, garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [`rclone`], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase verso un bucket remoto su **Backblaze B2** [`backblaze`]. L'automazione è gestita tramite un *cron job* che esegue il backup in modo incrementale ogni primo giorno del mese.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [`uptimerobot`]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, inviando notifiche immediate in caso di downtime.

4.4 Struttura dei dati e pulizia automatica

I dati ricevuti dalla board transitano nella collezione `data_sent_raw`, che funge da punto di ingresso grezzo. Ogni record viene immediatamente smistato nelle collezioni specializzate dall'hook server-side e successivamente eliminato nel blocco `finally` dell'hook stesso. Questo approccio garantisce che il database non accumuli dati ridondanti, mantenendo al contempo un audit trail completo nelle collezioni di destinazione. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat / lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);
- `battery_percent`: stato di carica della batteria (espresso in percentuale);
- `charging`: booleano che indica se la batteria è in corso di ricarica;
- `steps`: conteggio dei passi rilevato dall'accelerometro BMA400;
- `sleep`: booleano che indica se il dispositivo è in stato di inattività prolungata;
- `trip`: booleano che indica se l'animale si trova su un veicolo in movimento.

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile).

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules** basate su espressioni booleane:

- **List/View**: Visualizzazione permessa solo agli utenti autenticati.
- **Create**: Abilitata per il dispositivo LilyGo tramite autenticazione dedicata.
- **Update/Delete**: Ristrette ai soli amministratori del sistema.

4.6 Logica di Smistamento e Normalizzazione (Server-side Hooks)

Per ottimizzare la gestione dei dati e garantire la separazione delle responsabilità, è stata implementata una logica di smistamento automatico lato server tramite i *JavaScript Event Hooks* di PocketBase. L'hook `onRecordAfterCreateSuccess` si attiva immediatamente dopo il salvataggio di ogni pacchetto grezzo, smista i dati nelle collezioni specializzate e infine elimina il record da `data_sent_raw` nel blocco `finally`, garantendo la pulizia anche in caso di errori parziali.

4.6.1 Ottimizzazione delle query: lettura centralizzata della board

Il record board viene letto una sola volta all'inizio dell'hook tramite `getBoardRecord` e passato come parametro a tutte le funzioni che ne hanno bisogno (`computeStatus`, `saveBattery`, `notifyBoardUsers`), eliminando query ridondanti per ogni pacchetto ricevuto.

4.6.2 Funzionamento del Trigger `onRecordAfterCreateSuccess`

Il trigger elabora ogni pacchetto in cinque fasi sequenziali:

- **Batteria:** salva il record `battery_data` e controlla se inviare notifiche di cambio soglia.
- **Activity Tracking:** delega il calcolo del nuovo stato alla funzione `computeStatus` tramite il modulo `activity_manager.js` e gestisce la macchina a stati delle sessioni.
- **Posizioni:** salva le coordinate GPS agganciandole all'activity determinata al punto precedente, solo se le coordinate sono valide e l'activity non è stata scartata.
- **Pulizia:** elimina il record da `data_sent_raw` nel blocco `finally`.

4.6.3 Macchina a stati delle attività

Il cuore della logica server-side è una macchina a stati che modella il comportamento dell'animale. Gli stati sono distinti in *attivi* (dispositivo sveglio) e *sleep* (dispositivo in Deep Sleep), con una corrispondenza biunivoca tra le due categorie.

Stato attivo	Stato sleep	Descrizione
i — inside	d	Animale all'interno della geofence
s — search	p	Ricerca attiva: animale fuori zona con allarme
w — walk	z	Animale in passeggiata fuori zona (senza allarme)
v — trip	a	Animale su veicolo in movimento (allarme disattivo)
r — trip	q	Animale su veicolo in movimento con allarme attivo

Tabella 1: Macchina a stati delle attività: corrispondenza tra stati attivi e sleep

Tutti gli stati sleep si comportano in modo **uniforme**: vengono sempre chiusi con `is_active = false`. La funzione `computeStatus` normalizza internamente qualsiasi stato sleep nel corrispondente stato attivo prima di qualsiasi valutazione, permettendo di passare direttamente il `closedStatus` della sessione precedente senza pre-elaborazione esterna. Questo garantisce che la logica trip (`steps==0` mantiene lo stato viaggio) funzioni correttamente anche dopo una fase di sleep — sia per "a" (normalizzato a "v") che per "q" (normalizzato a "r").

Lo stato "v" modella il viaggio con allarme *disattivo*, mentre lo stato "r" modella il viaggio con allarme *attivo*. La distinzione permette al frontend di segnalare scenari di

urgenza diversi (es. animale rubato sul veicolo) e alla logica di notifica di inviare messaggi contestualmente appropriati. Una transizione tra "v" e "r" (o viceversa) durante il viaggio — causata dal cambio del flag di allarme — genera la chiusura della sessione corrente e l'apertura di una nuova, garantendo la tracciabilità del cambio di contesto.

Le transizioni tra stati attivi sono governate dalla funzione `computeStatus` che, ricevuto un pacchetto, calcola il nuovo stato secondo la seguente priorità:

1. Se `trip = true` $\Rightarrow$ stato "v" se `alarm = false`, stato "r" se `alarm = true` (priorità assoluta).
2. Se il dispositivo era in "v" o "r" (o nei corrispondenti sleep "a"/"q", normalizzati) e `trip` torna `false` con `steps == 0` $\Rightarrow$ rimane nello stato viaggio corrente (ancora fermo dopo il viaggio).
3. Se il dispositivo era in "v" o "r" e `trip` torna `false` con `steps > 0` $\Rightarrow$ ricalcolo pulito della geofence.
4. Altrimenti $\Rightarrow$ la macchina a stati valuta la posizione rispetto alle geofence e l'eventuale allarme utente: "i" se dentro, "s" se fuori con allarme attivo, "w" se fuori senza allarme o senza geofence configurate.

4.6.4 Gestione del Risveglio e Deduplicazione delle Sessioni

Un aspetto critico della logica è la gestione del risveglio dal Deep Sleep. Quando il dispositivo si sveglia non trova una sessione attiva (quella precedente è stata chiusa allo sleep). Il codice cerca la sessione chiusa più recente (`recentClosed`) per ricavare il `prevStatus` da passare a `computeStatus`: questo è fondamentale per il trip-sleep, dove "a" viene normalizzato a "v" e "q" viene normalizzato a "r", così con `steps==0` lo stato viaggio viene mantenuto correttamente.

La query di risveglio cerca tutti gli stati sleep validi: a, q, d, p, z. Le sessioni chiuse con `anomaly = true` (chiuse dal watchdog) sono escluse e non vengono mai riattivate.

Il sistema distingue tre scenari:

- **Risveglio nello stesso stato:** se la sessione era stata chiusa in uno stato sleep e il nuovo stato attivo calcolato corrisponde allo stesso stato attivo di partenza, la sessione viene riattivata (`is_active = true`) senza limiti di tempo. Questo permette di collegare correttamente le attività attraverso i cicli sleep, anche dopo ore di inattività.
- **Risveglio con cambio di stato:** la sessione chiusa viene prima corretta riportando il suo status alla versione attiva (es. "d" $\rightarrow$ "i", "z" $\rightarrow$ "w"), così da garantire la correttezza del riepilogo storico. Poi viene aperta una nuova sessione con il nuovo stato calcolato.

Le sessioni chiuse con `anomaly = true` (chiuse dal watchdog) non vengono mai riattivate: viene sempre creata una nuova sessione, evitando che una sessione anomala venga ripresa come se nulla fosse.

4.6.5 Watchdog di Inattività e Split Notturmo

Il sistema include due *cron job* per garantire la coerenza delle sessioni nel tempo:

- **Watchdog (ogni minuto):** controlla che nessuna sessione rimanga aperta indefinitamente a causa di un'interruzione imprevista del dispositivo (perdita di segnale LTE, spegnimento improvviso). Se una sessione attiva non riceve aggiornamenti per più di 10 minuti, viene chiusa automaticamente impostando `is_active = false` e il flag `anomaly = true`, e viene inviata una notifica push agli utenti della board. Il watchdog non tocca mai le sessioni in stato sleep, che per design possono rimanere inattive a lungo.
- **Split di mezzanotte (23:59 ora italiana):** gestisce le sessioni che attraversano il cambio di giornata. Per ogni board, l'ultima sessione attiva viene chiusa e ne viene aperta una nuova identica a partire dalla mezzanotte italiana. I timestamp vengono salvati in UTC: in ora legale (CEST, UTC+2) la chiusura avviene alle 21:59:59 UTC e l'apertura alle 22:00:00 UTC; in ora solare (CET, UTC+1) rispettivamente alle 22:59:59 UTC e 23:00:00 UTC. L'offset è calcolato dinamicamente tramite la funzione `getItalyTime()`, che gestisce automaticamente il cambio ora legale/solare secondo le regole europee, senza offset hardcoded. Il cron è schedulato su entrambi gli orari UTC (59 21,22 * * *) e un controllo interno filtra il tick non pertinente. Le sessioni in stato sleep vengono corrette al corrispondente stato attivo prima della chiusura, garantendo la correttezza del riepilogo giornaliero. Il cron include inoltre una verifica watchdog prioritaria: sessioni anomale (inattive da più di 10 minuti) vengono ignorate senza essere duplicate nel giorno successivo.

4.6.6 Vantaggi Architetture e Corrispondenza IoTWF

Questa implementazione riflette i principi del Modello IoTWF in due punti chiave:

1. **Efficienza del Livello 3 (Edge):** La LilyGo invia un unico pacchetto “bulk”, riducendo il tempo di attività del modem LTE e preservando la batteria.
2. **Ottimizzazione del Livello 5 (Data Abstraction):** L'applicazione Flutter non deve processare dati grezzi; interroga API già normalizzate dal trigger, garantendo una visualizzazione fluida della cronologia.

5 Front-end e Applicazione Mobile

L'applicazione mobile è stata sviluppata in *Flutter* [**flutter**], un framework cross-platform che consente di produrre un'unica base di codice per Android e iOS. I test sono stati condotti **esclusivamente su ambiente Android** a causa di vincoli tecnici, tra cui l'assenza di postazioni di lavoro con macOS, requisito necessario per il processo di build in ambiente Apple, e la mancanza di una licenza *Apple Developer Program*, indispensabile per garantire la persistenza dell'applicazione su dispositivo fisico oltre il limite di sette giorni previsto per i profili gratuiti. La comunicazione con il backend avviene tramite il PocketBase SDK ufficiale per Dart, che gestisce le chiamate REST e i meccanismi di autenticazione.

5.1 Architettura a livelli

Il codice è strutturato in quattro livelli principali, per separare le diverse responsabilità:

- **Repositories:** accesso ai dati remoti tramite PocketBase SDK, fornendo alle schermate dati in tempo reale e pronti all'uso.
- **Models:** definiscono la struttura dei dati convertendo i record JSON del backend in oggetti Dart.
- **Services:** gestiscono le funzionalità di sistema e le integrazioni esterne (autenticazione, mappe, tracciamento GPS e notifiche).
- **Screens:** interfaccia utente che si limita a richiedere i dati ai repository e a reagire ai loro aggiornamenti.

Questa struttura garantisce un codice ordinato e facile da aggiornare, mantenendo la logica di funzionamento ben separata dall'interfaccia grafica.

5.2 Autenticazione e persistenza della sessione

Per permettere all'utente di restare collegato in modo permanente e sicuro, abbiamo esteso l'**AuthStore** nativo di PocketBase, che normalmente salva i dati solo temporaneamente in memoria. L'autenticazione è quindi gestita tramite un **SecureAuthStore** personalizzato che agisce come una *cassaforte digitale*, riceve il token (JSON Web Token) dal server al momento del login e lo archivia in forma cifrata all'interno del dispositivo tramite **flutter_secure_storage**. Questo sistema sfrutta i sistemi di protezione hardware del dispositivo (Keystore su Android e Keychain su iOS) per impedire accessi non autorizzati.

Al riavvio dell'app, il token viene riletto dallo storage sicuro e viene tentato un refresh automatico della sessione. In caso di successo, l'utente accede direttamente alla schermata principale senza dover reinserire le credenziali; in caso di token scaduto o non valido, lo store viene svuotato e l'utente viene reindirizzato alla schermata di login.

5.3 Splash Screen e Caricamento Parallelo

La schermata di avvio svolge un ruolo attivo nel precaricare i dati necessari alla home page. Una volta verificata l'autenticazione, vengono avviate in parallelo, le seguenti operazioni:

- recupero del profilo utente e dello stato dell'allarme;
- recupero dell'ultima posizione GPS registrata dal dispositivo;
- recupero delle attività del giorno corrente filtrate per **board_id**.

Il **board_id**, identificativo univoco della board LilyGo associata a un dispositivo per utente, viene risolto dinamicamente interrogando la collezione **boards** sul backend tramite repository. Questo permette all'app di trovare il dispositivo corretto automaticamente, evitando di dover inserire il codice manualmente nel software.

5.4 Comunicazione in Tempo Reale

Completata la fase iniziale di caricamento dati, l'app stabilisce una **connessione persistente** al database, passando da una visualizzazione statica ad un monitoraggio dinamico.

Per le funzionalità che richiedono aggiornamenti continui come la posizione GPS, l'app sfrutta il meccanismo di *subscribe* offerto da PocketBase, basato su **Server-Sent Events**.

I repository (come `PositionsRepository`) espongono uno `Stream` tipizzato (ad esempio `Stream<Positions>`) a cui le schermate si sottoscrivono tramite una `StreamSubscription`.

In questo modo, ogni nuovo record inserito nel database viene notificato istantaneamente all'app. L'interfaccia utente si aggiorna dinamicamente evitando la tecnica del *polling*, che costringerebbe l'app a inviare continue richieste al server, ed eliminando la necessità di qualsiasi ricaricamento manuale della pagina da parte dell'utente.

Per garantire l'efficienza dell'applicativo e prevenire *memory leak*, ogni sottoscrizione viene rigorosamente interrotta tramite il comando `cancel()` nel metodo `dispose()` quando l'utente naviga verso un'altra schermata. Questa accortezza evita di mantenere attive connessioni inutili in background, ottimizzando l'uso della batteria e del traffico dati.

5.5 Geofencing e Geocoding

La logica di geofencing è implementata anche dal lato client tramite algoritmo *Ray-Casting*. Dato un punto GPS e un poligono definito da una lista di vertici, l'algoritmo traccia un raggio orizzontale dal punto verso l'infinito e conta le intersezioni con i lati del poligono, se il numero è dispari indica che il punto è interno al poligono, altrimenti è esterno. Questa implementazione sul dispositivo evita un sovraccarico di rete verso il server e permette di gestire le funzionalità visive in tempo reale, in totale autonomia dal *backend*.

Le zone vengono recuperate dalla collezione `geofences` su PocketBase, dove ogni record memorizza il nome della zona, i dati dell'indirizzo e la lista dei vertici del poligono. Solo le zone con il flag `is_active` impostato a `true` vengono considerate nel calcolo. Il risultato della verifica è una stringa con il nome della zona di appartenenza, oppure *"Fuori zona sicura"* se il punto non rientra in nessuna area attiva.

L'app integra un servizio di **geocoding** basato sulle API di *Nominatim* [**nominatim**], un motore di ricerca che interroga il database geografico open-source di OpenStreetMap per consentire:

- **Reverse Geocoding**: tradurre le coordinate GPS in un indirizzo testuale completo (Via, Civico, CAP e Città) per facilitare la lettura all'utente;
- **Forward Geocoding**: consentire all'utente di creare nuove zone inserendo un indirizzo testuale, che il sistema converte automaticamente in coordinate geografiche per posizionare correttamente il poligono sulla mappa.

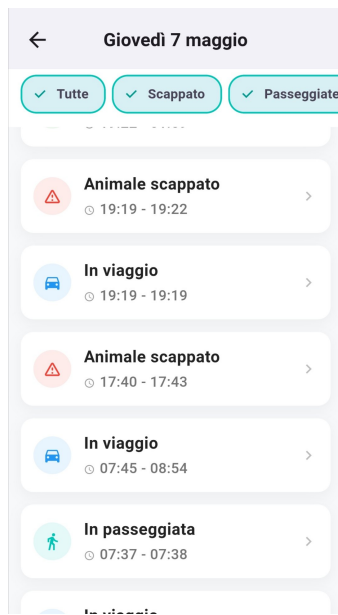
5.6 Schermate e Funzionalità Principali

5.6.1 Analisi delle Attività

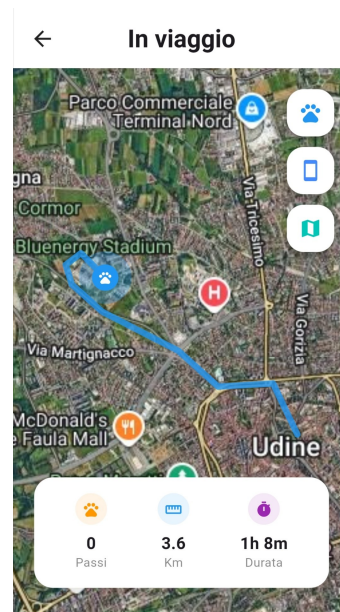
L'architettura dell'applicazione ruota attorno alla dashboard principale in `home.dart`, che funge da centro di controllo dinamico per il monitoraggio dell'animale, mostrandone l'ultima posizione rilevata. Un'altra caratteristica importante della pagina è l'interruttore dell'allarme. Se il sistema rileva che l'animale è in stato di fuga, l'interfaccia attiva automaticamente un blocco di sicurezza per impedirne la disattivazione. Inoltre, poiché lo stato dell'allarme si basa sulla variabile condivisa `_isUpdatingAlarm`, l'integrità dell'operazione è garantita da un **mutex** che congela l'input dell'utente durante la sincronizzazione asincrona con il database. Infine, la schermata raccoglie le statistiche quotidiane

come passi, chilometri e tempo di attività dell'animale, permettendo la consultazione dei dati storici tramite un selettore di data.

La visualizzazione dei dettagli delle attività di una giornata prosegue in `daily_recap.dart`, che raccoglie e organizza in ordine cronologico ogni monitoraggio visualizzato nella dashboard, con la possibilità di filtrare per specifiche attività. Selezionando una determinata sessione dal riepilogo, si accede alla schermata `activity_details.dart`, che mostra il tracciato specifico del percorso tramite una scia continua sulla mappa. Il sistema utilizza la funzione `CameraFit` per centrare l'inquadratura all'apertura, garantendo l'immediata visibilità dell'intero tragitto. Inoltre, la schermata ripropone le statistiche di passi, chilometri e durata, ricalcolate specificamente per l'attività selezionata.



(a) Visualizzazione di tutte le attività.



(b) Dettagli di un'attività.

5.6.2 Mappe in Tempo Reale

L'app utilizza mappe interattive sviluppate con `flutter_map`, permettendo di localizzare utente e animale in una visuale satellitare o stradale. La libreria viene ottimizzata tramite l'uso dei tile di `OpenStreetMap`, ovvero piccoli tasselli d'immagine caricati dinamicamente solo per l'area inquadrata, riducendo drasticamente il consumo di dati. Per preservare la batteria dello smartphone, le posizioni si aggiornano in tempo reale solo quando c'è un vero movimento, inoltre l'utente viene localizzato solo se ha dato il permesso.

Il sistema di controllo geografico si articola in due schermate principali che si scambiano dinamicamente in base allo stato dell'allarme. Quando l'allarme è disattivato, l'utente accede al sistema di `geofencing.dart`, un robusto strumento di sicurezza preventiva che permette di definire e gestire i confini virtuali rappresentati come poligoni sulla mappa.

All'attivazione dell'allarme, la schermata `geofencing.dart` passa a `tracking.dart`. In questa fase, il sistema adotta un approccio proattivo di sicurezza implementando un'interfaccia adattiva che cambia tra una modalità di monitoraggio e una di "inseguimento" qualora l'animale lasci la sua zona sicura attiva. Sulla mappa inoltre viene renderizzata una scia storica delle ultime posizioni registrate, fornendo un percorso visivo immediato che aiuta a comprendere la direzione del movimento recente dell'animale.

5.6.3 Gestione delle Zone Sicure

Dopo l’inserimento di un indirizzo (manuale o GPS) o la selezione di una zona dalla pagina di geofencing, la creazione e modifica delle aree avviene tramite `polygon_editor.dart`. L’utente può definire il perimetro toccando lo schermo per aggiungere i vertici. L’interfaccia è progettata per essere estremamente flessibile, consentendo all’utente non solo di aggiungere o annullare i vertici, ma anche di rifinirne la forma trascinando i punti già posizionati. Oltre a richiedere un minimo di tre vertici per formare una figura geometrica chiusa, un’area può essere creata solo se rispetta due controlli di validità:

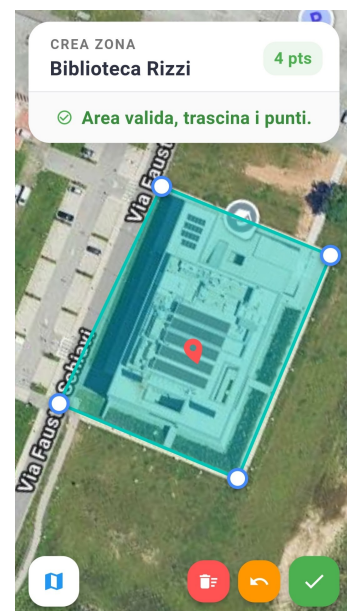
- Il **baricentro dell’area** non deve essere troppo distante dall’indirizzo selezionato, impedendo così la creazione di zone a più di 50 metri dal segnaposto reale.
- Non deve esserci alcuna **sovrapposizione con zone già esistenti** al fine di garantire l’univocità dei perimetri ed evitare conflitti logici durante il tracciamento.



(a) Scelta dell’inserimento.



(b) Compilazione indirizzo.



(c) Disegno dell’area.

6 Sistema di Notifiche Push

L’implementazione di un sistema di notifiche push è fondamentale per un dispositivo di tracciamento animali, in quanto permette di avvisare tempestivamente l’utente in caso di pericoli (uscita dal perimetro sicuro) o problemi tecnici (batteria scarica).

6.1 Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase

Durante la fase di sviluppo, è emerso un vincolo tecnico significativo riguardante l’integrazione tra il backend PocketBase e l’Sdk ufficiale di Firebase. PocketBase è scritto in linguaggio Go e utilizza **Goja** come motore di esecuzione JavaScript per la gestione della logica server-side (hooks). Goja è un’implementazione di ECMAScript 5.1 che presenta limitazioni strutturali rispetto a un ambiente Node.js `[nodejs]` standard:

- **Mancanza di moduli nativi:** Goja non supporta i moduli core di Node.js (come `fs`, `crypto` o `net`), indispensabili per il funzionamento dell'SDK di Firebase.
- **Runtime isolato:** L'SDK `firebase-admin` [`firebase_admin`] richiede un ambiente di esecuzione completo per gestire l'autenticazione tramite Service Account e le connessioni persistenti ai server Google.

Per superare queste limitazioni, è stata adottata un'architettura a **Bridge**, separando la logica di business dall'invio fisico dei messaggi.

6.2 Architettura del FCM Bridge

Il sistema si basa su un microservizio esterno sviluppato in Node.js [`nodejs`], denominato *FCM Bridge*. L'interazione avviene secondo il seguente flusso:

1. **PocketBase (Goja):** Rileva un evento (es. batteria scarica) e invia una richiesta HTTP POST al bridge tramite il modulo interno `$http.send`.
2. **FCM Bridge (Node.js):** Riceve la richiesta, carica il Service Account di Firebase e inoltra la notifica tramite l'SDK ufficiale [`firebase_admin`].
3. **Feedback e pulizia token:** Il bridge restituisce lo stato della consegna. In caso di errore 404 (token non registrato, app disinstallata) o 400 (token malformato), il backend provvede alla rimozione automatica del token dal profilo utente. I token FCM sono memorizzati come array JSON per supportare più dispositivi per singolo utente.

6.3 Logica di Invio e Trigger

Le notifiche vengono scatenate dai seguenti eventi, ciascuno con logica contestuale per evitare invii ridondanti:

Evento	Condizione di trigger	Nota
Uscita dalla zona	$i \rightarrow s$ oppure $i \rightarrow w$	Dipende dalla presenza di allarme utente
Rientro in zona	$s \rightarrow i$ oppure $w \rightarrow i$	
Allarme da passeggiata	$w \rightarrow s$	Allarme riattivato mentre fuori zona
Inizio viaggio confermato	$\text{qualsiasi} \rightarrow v/r$	Dipende dallo stato dell'allarme
Fine viaggio in zona	$v/r \rightarrow i$	
Fine viaggio fuori zona (allarme)	$v/r \rightarrow s$	
Fine viaggio fuori zona (libero)	$v/r \rightarrow w$	
Nessuna geofence	$i \text{ o } s \rightarrow w$	Zone disattivate
Batteria bassa	$\leq 20\%$ (cambio soglia)	Solo al cambio di fascia
Batteria critica	$\leq 10\%$ (cambio soglia)	Solo al cambio di fascia
Batteria in carica	<code>charging</code> passa a <code>true</code>	Solo al cambio di stato
Watchdog	Silenzi > 10 minuti	Chiusura automatica sessione

Tabella 2: Trigger del sistema di notifiche push

6.4 Permessi e Localizzazione

Per abilitare la ricezione delle notifiche lato client, è stato fondamentale integrare il file di configurazione `google-services.json`, generato direttamente dalla console di Firebase e inserito all'interno della directory `android/app` del codice sorgente. Questo file contiene le chiavi API, l'identificativo del progetto (`Project ID`) e i dettagli di configurazione necessari per autenticare in modo sicuro l'applicazione mobile con i servizi Google, senza esporre le credenziali nel codice in chiaro. Durante la fase di build, il *plugin Gradle* di Google Services, che si occupa di mappare i parametri di configurazione nel codice Android, processa questo file per inizializzare automaticamente l'SDK di Firebase all'avvio dell'app, abilitando così la connessione sicura a FCM e la generazione del token univoco del dispositivo. Quest'ultimo viene dapprima salvato localmente e poi sincronizzato con il profilo utente su PocketBase all'interno del campo `tokenFCM`. Tale campo è modellato come una lista per permettere all'utente di ricevere le allerte su più dispositivi contemporaneamente; prima di ogni salvataggio, un controllo di duplicazione verifica che il *token* non sia già presente, evitando così la ricezione di notifiche ridondanti.

Dal punto di vista dell'interazione utente, l'attivazione dei servizi, sia di notifica che di tracciamento, richiede una corretta gestione dei permessi di sistema. Al primo avvio l'applicazione mostra un *dialog* informativo che spiega le finalità del monitoraggio geografico dell'utente prima di attivare la richiesta di autorizzazione. Acquisito il consenso, il software verifica l'attivazione del modulo GPS e se entrambi i requisiti sono soddisfatti, vengono recuperate le coordinate. Subito dopo, con un approccio del tutto analogo, viene presentata la richiesta per le notifiche push, passaggio indispensabile per autorizzare la generazione del *token FCM* descritto in precedenza. Infine, per evitare di riproporre il *popup* ai successivi avvii, viene salvata localmente tramite `SharedPreferences` un'apposita variabile booleana che funge da indicatore dell'avvenuta lettura dell'informativa.

7 Vincoli di Sistema

La progettazione e l'ingegnerizzazione della soluzione sono state guidate e delimitate da una serie di vincoli tecnologici, fisici e d'infrastruttura. Tali vincoli definiscono il perimetro operativo entro cui il sistema garantisce la consistenza dei dati e la continuità del servizio.

7.1 Vincoli Hardware e Ambientali

- **Dipendenza dalla Rete di Trasmissione Cella (LTE):** Il flusso di telemetria dal dispositivo al backend è strettamente vincolato alla presenza di copertura radio-mobile. In assenza di segnale cellulare, la trasmissione sincrona dei pacchetti non è garantita. Questo vincolo richiede che la logica del firmware sia progettata per gestire code di trasmissione e buffer locali onde evitare la perdita definitiva dei dati raccolti in zone isolate.
- **Degradazione del Segnale GNSS in Ambienti Indoor:** L'accuratezza del posizionamento è vincolata alla visibilità diretta della costellazione satellitare (*Line-of-Sight*). In contesti indoor, lo scattering e l'attenuazione strutturale rendono il modulo GNSS altamente instabile o impossibilitato al fix. Il backend deve pertanto isolare e scartare i pacchetti con metadati di accuratezza insufficiente per non alterare le metriche di tracciamento.

- **Budget Energetico del Dispositivo IoT:** Il modem LTE rappresenta il fattore di consumo più critico dell'intero hardware. La frequenza e la durata delle sessioni di trasmissione sono fortemente vincolate alla capacità della batteria. L'architettura software del backend e i tempi di polling del watchdog sono stati calibrati attorno a questo vincolo, accettando un compromesso (*trade-off*) sulla granularità temporale dei dati in favore di una maggiore autonomia complessiva.

7.2 Vincoli Architettureali e di Concorrenza

- **Persistenza e Hosting in Ambiente Locale:** L'adozione di un'istanza locale di PocketBase, esposta sulla rete pubblica tramite un tunnel **ngrok**, introduce vincoli di scalabilità verticale e dipendenza da servizi di terze parti. L'architettura è vincolata ai limiti di banda e alle policy di *rate-limiting* del piano di tunneling gratuito. Questo assetto definisce un vincolo ideale per scenari di prototipazione, ma esclude la scalabilità multi-nodo senza una migrazione verso un'infrastruttura cloud dedicata.
- **Race Condition sulle Transazioni Concorrenti:** Il framework PocketBase gestisce l'accesso a SQLite astraendo parzialmente le transazioni atomiche isolate sui singoli record a livello applicativo JavaScript. Sotto stress o in caso di ricezione quasi simultanea di due pacchetti dalla medesima scheda (con un delta temporale inferiore al tempo di scrittura sul database), si genera un vincolo di concorrenza (*race condition*). Gli scenari critici legati a questo vincolo includono:
 1. *Inconsistenza nelle transizioni:* Il secondo pacchetto legge lo stato attivo prima che il primo ne completi la chiusura, tentando di sovrascrivere un record già invalidato.
 2. *Duplicazione delle sessioni attive:* Durante un risveglio simultaneo dal **Deep Sleep**, entrambi i flussi potrebbero intercettare il medesimo record **sleep**, istanziando due sessioni attive concorrenti e violando l'invariante fondamentale della macchina a stati.

Questo vincolo architetturale è stato mitigato e accettato in fase di design, poiché la frequenza nativa di invio dei pacchetti (pari o superiore a 30 secondi) rende l'accadimento dell'evento statisticamente trascurabile nell'esercizio ordinario.

7.3 Vincoli di Integrazione e Dati Esterni

- **Completezza dei Metadati Geografici Crowdsourced:** Il servizio di geocodifica inversa è vincolato alla densità e alla qualità dei dati presenti su **OpenStreetMap**. Nelle aree rurali o nei centri abitati di minori dimensioni, l'assenza di tag strutturati (quali numeri civici, CAP o precise denominazioni stradali) inficia la precisione delle query inviate all'API **Nominatim**. Poiché quest'ultimo motore adotta un vincolo di corrispondenza rigida (*strict matching*) sui parametri, l'incompletezza del dato geografico di origine si traduce inevitabilmente in un fallimento della risoluzione dell'indirizzo leggibile, delimitando l'efficacia della visualizzazione testuale lato client.

8 Implementazioni Future

Il progetto nel suo percorso di sviluppo ha trovato delle possibilità future di scalabilità dell'App, di seguito le idee che abbiamo accavallato per procedere nella migliore implementazione delle precedenti funzioni descritte.

8.1 Stato della batteria

Per garantire all'utente un corretto valore della batteria aggiornato sugli ultimi pacchetti è richiesto un calcolo con dati storici, questo è uno dei possibili ulteriori passaggi che può essere sviluppato all'interno dell'applicazione per quanto riguarda i dati da fornire all'utente. Non risulta come attività critica in quanto il sistema riesce a garantire all'utente una notifica nel momento in cui la batteria raggiunge (e scende) sotto il livello del 20%.

8.2 Database, Infrastruttura Cloud e Gestione della Concorrenza

Il passaggio da un ambiente di prototipazione locale a una soluzione orientata alla produzione prevede la migrazione dell'intero backend su un'infrastruttura Cloud dedicata (PaaS o VPS come AWS, Google Cloud o DigitalOcean). Questa evoluzione consentirà di superare i vincoli attuali attraverso i seguenti interventi strutturali:

- **Assegnazione di Dominio e Sicurezza TLS:** Sostituendo il tunnel temporaneo **ngrok** con un dominio di secondo livello proprietario, protetto da certificati SSL/TLS (es. Let's Encrypt), si garantirà una connessione HTTPS/WSS nativa, stabile e priva di interruzioni di sessione o rate-limiting arbitrari.
- **Risoluzione delle Race Condition via Message Broker:** Per azzerare il rischio di collisione nell'aggiornamento simultaneo delle attività (derivante dalla ricezione di pacchetti ravvicinati), si prevede l'introduzione di un middleware di messaggistica asincrona, come **Redis** o **RabbitMQ**, interposto tra l'endpoint di ricezione e il database. Questo livello fungerà da coda FIFO (*First-In, First-Out*), serializzando i pacchetti provenienti dallo stesso tracker ed eseguendo le transazioni in modo strettamente sequenziale, preservando così l'invariante del sistema.
- **Migrazione del DBMS:** Qualora il volume di dispositivi connessi dovesse saturare le capacità di lock di SQLite, l'architettura consentirà di scalare verticalmente migrando verso un database relazionale centralizzato (come **PostgreSQL**), nativamente ottimizzato per transazioni ACID altamente concorrenti.

8.3 Ingegnerizzazione e Costruzione della Scheda Hardware

L'attuale prototipo hardware, sebbene funzionale alla validazione del software, richiede una fase di ingegnerizzazione elettronica per poter essere trasformato in un dispositivo commerciale realmente indossabile dall'animale. Lo sviluppo futuro si articolerà su tre assi principali:

- **Sviluppo di un PCB Custom:** Progettazione di un circuito stampato personalizzato (*Printed Circuit Board*) mediante strumenti EDA (come KiCad o Altium Designer). L'integrazione del microcontrollore, del modem LTE e del modulo GNSS

su un'unica scheda consentirà di ridurre drasticamente il fattore di forma (*form factor*), minimizzando l'ingombro e il peso del tracker sul collare dell'animale.

- **Ottimizzazione del Power Management:** Introduzione di un circuito integrato dedicato alla gestione dell'energia (*PMIC*) e di un chip di *fuel gauge* hardware (es. serie MAX17043). Questo permetterà di implementare il calcolo predittivo basato sui dati storici menzionato nella sezione precedente, offrendo letture della percentuale di carica estremamente precise e non inficiate dai picchi di assorbimento del modem durante la trasmissione.
- **Industrial Design e Protezione Ambientale:** Progettazione di un *enclosure* (involucro) personalizzato tramite modellazione 3D e stampaggio a iniezione. La scocca dovrà garantire un grado di protezione minimo pari a **IP67** o **IP68**, rendendo il dispositivo totalmente impermeabile all'acqua, al fango e resistente agli urti causati dai movimenti repentini dell'animale all'aperto.

9 Conclusioni